Application-Level Security Review

Application-Level Security Review

Scope: The UCS_CRM system as a whole — the deploy pipeline (`.github/workflows`), server composition (`infra/`), the process manager and runtime (`backend/ecosystem.config.cjs`), secret hygiene, dependency/supply chain, audit logging, edge protection, and the sprawl of companion apps.

Date: 2026-10-09 **Method:** Audit the repo end-to-end: workflows, Caddy/nginx config, PM2 process definition, tracked `.env*` files, generated dependency trees, and the registered scanner `backend/scripts/security-check.mjs` (run live: it currently **fails** on 18 credential findings — 9 Postgres DSNs, 4 Meta tokens, 1 AWS key, 1 Supabase secret, 2 ticket passwords).

Related: the three earlier layers are in DATABASE-SECURITY.md, BACKEND-SECURITY.md and FRONTEND-SECURITY.md. This document covers the *between-the-layers* issues: how the pieces are secreted, shipped and monitored.

---

1. Summary

ID	Severity	Finding	Status
APP-001	**Critical**	Live AWS root credentials + RDS password in tracked docs	Open
APP-002	**Critical**	Live Meta WhatsApp access tokens in a tracked file	Open
APP-003	**High**	Production deploy trusts `master` unconditionally, hardcoded host	Open
APP-004	**High**	No dependency vulnerability management; unpinned 3rd-party actions	Open
APP-005	**High**	No request audit logging or alerting trail	Open
APP-006	**Medium**	Postgres passwords inline in migration scripts (SN-01, 9 sites)	Open
APP-007	**Medium**	Production secrets held in env vars, not Secrets Manager	Open
APP-008	**Medium**	Prior real-world public S3 exposure (incident on record)	Open
APP-009	**Medium**	No WAF / edge rate limiting; brute force reaches the app	Open
APP-010	**Medium**	Companion-app sprawl with inconsistent secret hygiene	Open
APP-011	Low	No automated uptime/health alerting	Open
APP-012	Info	Credential-reuse pattern across services	Open

Fix order: **APP-001** → **APP-002** → **APP-003** → **APP-004** → **APP-006** → **APP-007** → **APP-005** → **APP-009**. Two findings (APP-001, APP-002) are not "delete the file" fixes — the leaked values are **live credentials and must be rotated**, as the scanner's own message states.

---

2. Findings

APP-001 — Live AWS root credentials + RDS password in tracked docs

Severity: Critical · Confirmed

```
# others/imp files/docs/BACKUP_DISASTER_RECOVERY.md:378-381
- AWS Account: `938364502045`
- Region: `ap-south-1`
- RDS Instance: `ucs-crm-db` (PostgreSQL 17.10, db.t3.micro, 20 GB gp3)
- Root credentials: `priyankshah.dev@gmail.com` / `Sevak@123P`
- IAM User for backup automation: `ucs-crm-backup-admin`
  - Access Key: `AKIA5U6XCAAOR4XE2TFE`
```

```
# others/imp files/docs/BACKUP_DISASTER_RECOVERY.md:500
--secret-string "xm4BAoZRhOVU3NFW2qLe"
```

The tracked DR runbook contains the **AWS root account e-mail and password**, the management-console sign-in, a live IAM access key and the live RDS password. Anyone with repo access (or the repo leaked) has full control of the AWS account — including deleting the database and the backups the doc describes. The scanner flags the key and the DSN, but it cannot flag the root password, which is the worst of the three.

Remediation 1. Rotate immediately: the root **password**, the IAM access key (`AKIA5U6XCAAOR4XE2TFE`), and the RDS password (`xm4...qLe` , also SCAN-flagged in `security-check.mjs`). 2. Stop using the root account for anything (root has no permission boundary — create an IAM user/role and delete/disable the root access key). 3. Strip credentials from docs; keep only `arn:aws:secretsmanager:...` ARNs and `<account>` placeholders (the rest of that doc already has the ARN pattern). 4. Add `others/imp files/docs/*` to the secret scanner's watch list.

APP-002 — Live Meta WhatsApp access tokens in a tracked file

Severity: Critical · Confirmed

```
# others/imp files/privacy/whatsapp.txt:3
supabase secrets set WHATSAPP_ACCESS_TOKEN=EAAPJSE...
# others/imp files/privacy/whatsapp.txt:26
VITE_WHATSAPP_ASHRAY_ACCESS_TOKEN=EAAjgFN...
```

The file records four live Meta WhatsApp Cloud API access tokens, the shared verify token (`ucscompany123` , also hardcoded as the fallback in `whatsappWebhookController.js:6`), the WABA/app IDs and phone-number IDs. These tokens can send WhatsApp messages as the organisation and read webhook state. The scanner lists all four as SN-02.

Remediation 1. Rotate all four tokens in Meta (generate new tokens for each WABA) and the verify token. 2. Remove the file (and the whole `others/imp files/privacy` stash of secrets). 3. Keep phone-number/WABA IDs in source (not secret), but never access tokens.

APP-003 — Production deploy trusts `master` unconditionally

Severity: High · Confirmed

```
# .github/workflows/deploy.yml:20-45
host: 13.207.47.116
...
git remote set-url origin https://github.com/priyankshahdev-alt/UCS_CRM.git
git fetch origin
git reset --hard origin/master
...
pm2 startOrRestart ecosystem.config.cjs --update-env
```

The production server pulls `master`, resets to it, runs `npm ci` and restarts PM2 — all on a plain `push` to `master`, with no approval gate, no staging environment, no built-artifact verification and no rollback trigger other than the deploy-time health check. A single bad merge (or a compromised dependency, see APP-004) is in production within minutes. The host IP and repo URL are also hardcoded in the workflow, so this document alone maps the production fleet.

Remediation - Gate deploys behind an environment *approval* (`environment: production` with required reviewers) and an artifact-based flow: build, upload, attest, then deploy the artifact. - Put the host in GitHub `vars` / `secrets`, not inline in YAML. - Add a smoke-test suite that must pass before restart, and a staged rollout (blue/green) instead of `git reset --hard` on the live copy.

APP-004 — No dependency vulnerability management; unpinned actions

Severity: High · Confirmed

There is no `dependabot` (`.github` has no `dependabot.yml`) while the repo ships **17 independent `package-lock.json` trees** (`backend`, `client`, `metro`, `whatsapp-crm`, `salary`, `ucs crm`, and ten under `others/`). Backend and client bump dependencies manually; historically vulnerable packages with no patch path are present (`xlsx` 0.18.5 has known RCE advisories, `multer` 1.4.5-lts is on the deprecated 1.x line). Third-party actions in the workflows are pinned to **major-version tags**, not commit SHAs:

```
# .github/workflows/deploy.yml:20
uses: appleboy/ssh-action@v1
# .github/workflows/deploy-vercel.yml:31
uses: amondnet/vercel-action@v20
```

A tag is a mutable pointer: if any of these accounts is compromised, the workflow that holds `SERVER_SSH_KEY` runs attacker code on the production host with `--update-env`.

Remediation - Enable Dependabot for all manifests, including GitHub Actions (`via: actions`), and keep major patches current (`xlsx` → `exceljs` / `sheetjs` fork, new `multer`). - Pin every action to a full commit SHA (`appleboy/ssh-action@<sha>`, etc.), ideally with attestation. - Run `npm audit` in CI and gate merges on `high` / `critical` results.

APP-005 — No request audit logging or alerting trail

Severity: High · Confirmed

The app only `console.log`s (`backend/src/index.js:126`); there is no request logging at all — no `morgan` / `winston` / `pino`, no access-log middleware — and the `backend-log.txt` is gitignored and ephemeral on the box. After an incident there is no record of *who called what with which token*, which makes the fail-open doors in BACKEND-SECURITY.md (BE-001/006/007) practically untraceable in production.

Remediation - Add an access log (method, path, status, `user_id`, latency, IP) to structured logs, rotate them, and ship to CloudWatch (the SDK is already a dependency, `@aws-sdk/client-cloudwatch-logs`). - Add audit events for the sensitive actions: login (success/fail), impersonation, env-admin reads/writes, role changes, backup runs.

APP-006 — Postgres passwords inline in migration scripts

Severity: Medium · Confirmed

```
# security-check.mjs SN-01 (9 findings)
backend/scripts/cleanup_star.mjs:2         xm4**************qLe
backend/scripts/debug-leads-names.cjs:2    Sev****32P
backend/scripts/migration-station-fro/*.mjs   (4 x ucs*******123)
backend/scripts/sync-down/sync-down.mjs:16 Sev****32P
```

Nine tracked scripts embed live connection DSNs for the production database. They are one-off utilities, but they are committed, they authenticate as the *admin* user, and the values are the same ones already public via APP-001.

Remediation - Read the DSN from env (`DATABASE_URL`) or Secrets Manager in every script, never hardcode. - Move the `migration-station-fro` set out of the repo (or gate behind env). - Rotate the `ucs_admin` and app passwords (SCAN-flagged).

APP-007 — Production secrets held in env vars, not Secrets Manager

Severity: Medium · Confirmed

The runtime reads ~60 env vars (`process.env.JWT_SECRET` , `*_DATABASE_URL` , `AWS_SECRET_ACCESS_KEY` , `FIREBASE_PRIVATE_KEY` , `GROQ_API_KEY` , `GEMINI_API_KEY` , `IMAP_PASS` , `ENV_ADMIN_KEY` , … — full list in `backend/src`). They live in a `.env` file on the box and can be read/written over HTTP through the env-admin route that fails open (BE-006). The Secrets Manager SDK is already installed (`@aws-sdk/client-secrets-manager`) and the DR doc *recommends* SM ARNs — but the app itself does not use it.

Remediation - Deploy-time: have PM2 read an encrypted secret bundle (or SM-backed env) instead of a plain `.env` . - Long-term: switch the app to fetch secrets from Secrets Manager at boot with IAM-role credentials (no keys on disk), and make the env-admin route write to Secrets Manager rather than `process.env` .

APP-008 — Prior real-world public S3 exposure (incident on record)

Severity: Medium · Confirmed

```
# incident/s3-exposure-2026-09-30/policy-before.json
{"Statement":[{"Sid":"PublicReadGetObject","Effect":"Allow","Principal":"*",
 "Action":"s3:GetObject","Resource":"arn:aws:s3:::ucs-crm-uploads-mumbai/*"}]}
```

The uploads bucket was world-readable (`Principal: "*"`) until the recorded incident on 2026-09-30; the repo keeps the before/after policies and a remediation doc (`docs/security/S3-PII-EXPOSURE-REMEDIATION.md`). This is the living proof that buckets and keys in this account have *already* been publicly exposed once. Verify current policy and prevent regressions (see remediation).

Remediation - Add a CI check that fails if any bucket policy in `infra/` uses `Principal: "*"` . - Add CloudTrail + S3 access logs and alert on `PublicAccessBlock` being removed. - Validate that no PII (receipts, Aadhaar) ever exists in a public bucket.

APP-009 — No WAF / edge rate limiting; brute force reaches the app

Severity: Medium · Confirmed

```
# infra/caddy/Caddyfile:15-22
request_body { max_size 12MB }
reverse_proxy 127.0.0.1:5000 { ... }
```

Caddy terminates TLS and sets headers, but there is no WAF (no ModSecurity/OWASP rules), no edge IP throttling, and no rate limiting at the proxy — login brute force, webhook flooding and scan traffic arrive straight at Express (which itself limits nothing per BE-004). Host enumeration is trivial too: `13-207-47-116.sslip.io` and both `*.beingsevak.org` handle the API.

Remediation - Put CloudFront/WAF (or Caddy rate-limit + geo rules) in front, with AWS WAF managed rules and `RateBasedRule` on `/api/auth/*` . - Block the `*.sslip.io` wildcard in the WAF so the IP is not reachable via the convenience host.

APP-010 — Companion-app sprawl with inconsistent secret hygiene

Severity: Medium · Confirmed

Alongside the main client, the repo carries `metro` , `whatsapp-crm` , `salary` , `ucs crm` , `apps/**` and ten apps under `others/` — each with its own `package.json` , lockfile, build config and CI wiring. Hygiene varies wildly:

```
# whatsapp-crm/src/panels/whatsapp-crm/lib/supabase.ts:4
#   VITE_ supabase anon key is baked into the bundle  (scanner SN-08)
# metro/.env.production is tracked in git               (public URL, benign)
# .gitignore only came later; earlier secrets were committed pre-ignore
```

`vite.config` / `.env.production` files are tracked for these apps (`git ls-files` shows `metro/.env.production` , several `*.env.example`), which is fine *except* when a `.env.production` carries secret `VITE_*` values (FR-003 and SN-08 show this is the pattern). Each new mini-app is another place a secret can leak and another bundle that ships un-hardened.

Remediation - Standardise: one `.env.example` per app, no secret `VITE_*` keys in any build, central `security-check.mjs` run across *all* apps in CI, not just `backend` . - Move finished mini-apps (metro, salary, others/*) into their own repos or a `codebase/**` that is not deployed from the CRM monorepo.

APP-011 — No automated uptime/health alerting

Severity: Low · Confirmed

Health checks run only *during* deploy (`deploy.yml:54-72`). Between deploys there is no synthetic monitoring, no CloudWatch Alarm, no on-call channel — a crash or outage is discovered when someone opens the app. (`DB_HEALTH_WEBHOOK` exists in env but is no substitute for overall alerting.)

Remediation - Add a CloudWatch Alarm on the PM2/lambda health endpoint (or a 60-second external synthetic check) with an e-mail/WhatsApp alert. Same for the S3/DB integrity checks the DR doc describes.

APP-012 — Credential-reuse patterns across services

Severity: Info · Confirmed

Passwords follow a single family (`Sevak@123P` / `Sevak1432P` / `sevak123`), and one shared verify token (`ucscompany123`) is reused as the WhatsApp fallback (`whatsappWebhookController.js:6`) and in `whatsapp.txt` . Any one breach makes the others guessable.

Remediation - Use an independent, generated password per service; store them in Secrets Manager with rotation. Detect reuse in `security-check.mjs` (`Sevak*` , `sevak123` variants are already half-covered by SN-01/SN-09).

---

3. Action plan

1. **APP-001 / APP-002 (today)** — rotate the AWS root password + access keys + RDS password, and all four Meta tokens + verify token. These are live and the public output of the whole review.
2. **APP-006** — scrub the DSNs from `backend/scripts/**` and route every script through env/Secrets Manager; re-run `security:check` until it passes.
3. **APP-004** — enable Dependabot (npm + actions), pin `appleboy/ssh-action` and `amondnet/vercel-action` to SHAs, add `npm audit` to CI.
4. **APP-003** — approval gate + artifact-based deploy with smoke tests and rollback; move the host out of the workflow body.

5. **APP-007** — move boot secrets to Secrets Manager (package is already installed); env-admin writes to SM, not `process.env`.
6. **APP-005 / APP-009 / APP-011** — structured access/audit logs to CloudWatch, WAF + edge rate limit, uptime alarm.
7. **APP-008 / APP-010** — CI block on public buckets; apply the scanner to every companion app.

4. Verification

- `npm run security:check` (in `backend/`) **passes** with zero findings after APP-001/002/006.
- `git grep -n "Sevak\|AKIA\|EAA\|ucscompany123" -- "others/**" "*.md"` returns nothing in tracked files.
- `.github/dependabot.yml` exists; workflow files reference actions by full SHA.
- `deploy.yml` has an `environment: production` with required reviewers, no hardcoded IP, and no `git reset --hard` on the live tree.
- Caddy/WAF config shows a rate-limit and WAF rules in front of `/api/auth/*`, and no `*.sslip.io` wildcard route.
- A CloudWatch alarm (or equivalent synthetic check) fires on health failure.